



BITS Pilani
DIGITAL
Excellence made yours

- Presented by:
- Manish Rachakonda
(2023EBCS668)

TalkToJesus

- BSc Computer Science (2025-2026)

A Bilingual Voice-First AI Spiritual Companion

- Under the guidance of
- Swapnil Saurav

Problem Statement

Telugu Bible apps have millions of downloads - and zero conversation.

- One-way communication - they present content; none answer a question
 - Access is gated on a person - a pastor must be available, scheduled, approachable
 - Guidance is not situation-aware - a reading plan does not know what you are facing
 - Language and culture - general AI assistants are not scripture-grounded, and their
 - Telugu is not idiomatic - addressing a believer as "naa bidda" (నా బిడ్డ) is not
 - a translation problem
-
- The gap: a believer with a question at 2 a.m., in Telugu, has nowhere to ask it.

Objectives & Scope

Objectives (PO1-PO7):

- PO1-PO2 Cross-platform app + secure backend with auth, conversation, subscriptions
- PO3-PO4 LLM for scripture-grounded replies; voice in and voice out
- PO5 Bilingual English/Telugu with culturally correct address
- PO6-PO7 Subscription monetisation; containerised deployment on GCP

• Scope:

- In - conversation, Bible reader, hymn library, subscriptions, admin console
- Out - app store publication, video avatar, community features, RAG over scripture

Existing System / Literature Review

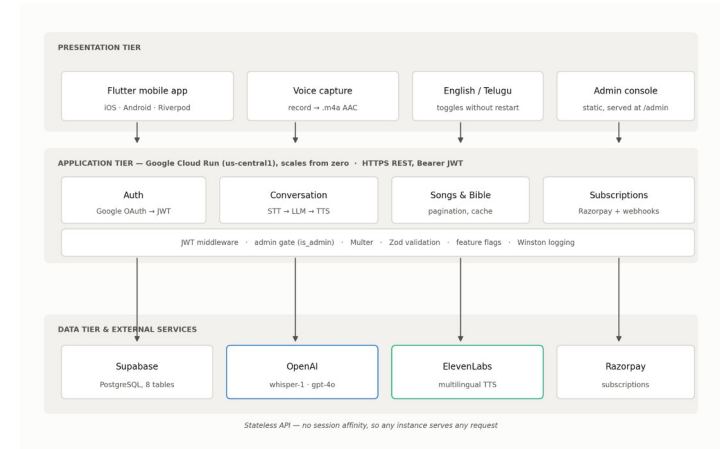
Conversational? Scripture-grounded? Telugu? Voice-first?

- YouVersion Bible App - strong content, no dialogue
 - Telugu Bible apps - Telugu and scripture, but static content only
 - Pray.com - guided prayer and audio, no conversation, no Telugu
 - General AI assistants - conversational, but not scripture-grounded, weak Telugu
 - Glorify / Abide - devotional, no conversational AI, no Telugu
-
- Each covers some. None covers all four. That intersection is the contribution.

Proposed System Architecture

Three tiers, one stateless API:

- Presentation - Flutter (iOS/Android, Riverpod) + the admin console
 - Application - Express 5 / TypeScript, 28 endpoints, Cloud Run, scales from zero
 - Data - Supabase PostgreSQL (8 tables) + OpenAI, ElevenLabs, Razorpay
-
- One decision drives the rest: the API holds a service-tier key that
 - bypasses row-level security - so authorization lives in application code.



Tools & Technologies

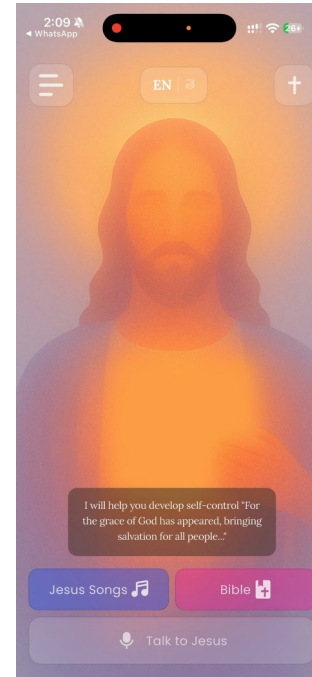
Language / Framework: Dart + Flutter 3.38.6 | TypeScript 5.9 + Express 5 on Node 20

- State / Validation: Riverpod 2.5 | Zod 4.1
- Database: Supabase (PostgreSQL), 8 tables, no ORM
- AI services: OpenAI whisper-1 (STT) + gpt-4o (LLM) | ElevenLabs (TTS)
- Payments / Identity: Razorpay subscriptions | Google OAuth 2.0 -> HS256 JWT
- Infrastructure: Docker, Cloud Build, Cloud Run (us-central1), Artifact Registry
- Observability: Winston, Sentry, PostHog, per-stage latency instrumentation
- Quality: Jest (66 tests) + flutter_test (75 tests) = 141, gated in CI

Implementation / Demo

LIVE DEMO

- 1. Sign in with Google
 - 2. Home screen - switch EN -> TE (తెలుగు), every label changes, no restart
 - 3. Bible reader in Telugu - Genesis 1, then change translation
 - 4. Hymn library - 12 bundled hymns, plays with no network
 - 5. Speak a question - record, send, hear the reply
 - 6. Free tier exhausted -> paywall -> Razorpay UPI -> badge turns active
 - 7. Conversation history - the full turn with scripture citations
 - 8. Admin console - live metrics, then the latency breakdown
-
- Fallback: the recorded demo, and the console's offline demo mode.



Results & Analysis

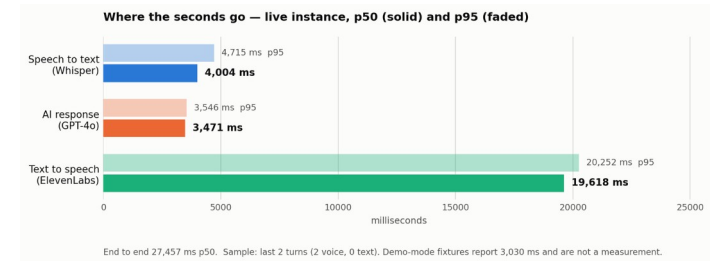
20/20 functional requirements implemented | 141 automated tests, 0 failing

- Full subscription flow verified end to end on device, real UPI payment

- **MEASURED pipeline latency (live instance, p50):**

- Speech to text (Whisper) 4,004 ms 15%
- AI response (GPT-4o) 3,471 ms 13%
- Text to speech (ElevenLabs) 19,618 ms 71%
- End to end 27,457 ms

- NFR1 required 5 seconds. Not met - and the instrumentation is what found it.



Challenges & Limitations

Latency fails NFR1 by ~5x - dominated by speech synthesis

- Authorization rests on application code - RLS is enabled but unpolicied and bypassed,
 - and no automated test asserts isolation between users
- Secrets committed - a long-lived token, PostHog key and Sentry DSN must be rotated
- No rate limiting; CORS is unrestricted
- Transcripts are personal data with no retention or deletion policy
- Coverage stops at the service layer - no widget, integration or end-to-end tests
- Three bundled hymns are CC BY-SA and need a credits screen the app does not have

Conclusion & Future Work

Conclusion:

- All 7 primary objectives met; 20/20 functional requirements implemented
- 141 automated tests passing across both tiers, gated in CI
- Bilingual voice conversation working in Telugu with culturally correct address

• Future Work:

- 1. Tighten the persona prompt and re-measure - likely the bulk of the 19.6 s
 - 2. Stream synthesised audio so playback starts before synthesis finishes
 - 3. Add authorization-isolation tests - the most valuable missing test
 - 4. Rotate the committed secrets; add rate limiting; restrict CORS
 - 5. Data retention policy; about/credits screen for CC BY-SA compliance
-
- The system measures itself - and that is how it caught its own claim.